

南開大學
Nankai University

计算机学院

计算机组成原理 lab1 实验报告

加法器的改进

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

2026 年 4 月 3 日

目录

1 实验目标 **2**

2 实验原理 **2**

3 实验过程 **4**

3.1 adder.v 模块的修改 4
3.2 仿真文件的编写与测试 4
3.3 lcd_module 模块的导入 5
3.4 顶层模块 adder_display.v 的改写 5
3.5 约束文件的修改 8

4 实验结果 **8**

4.1 波形仿真结果 8
4.2 波形数据分析 9
4.3 实验箱上板结果 11
4.4 实验箱结果分析 11

5 实验收获 **12**

1 实验目标

本实验要求在原始两个 32 位数加法器的基础上进行改进，实现四个 32 位数及 1 位低位进位输入的加法器设计。实验的主要目标如下：

1. 熟悉 LS-CPU-EXB-002 实验箱、Vivado 软件平台以及基于 FPGA 的基本设计流程；
2. 理解多操作数加法器的设计方法，在原始两个 32 位数加法器的基础上，改进实现四个 32 位数的加法器；
3. 修改 testbench 模块，对改进后的四操作数加法器进行行为级仿真，并通过波形验证设计功能的正确性；
4. 修改 display 模块和约束文件，使设计能够调用 LCD 触摸屏完成数据输入与结果显示，并在实验箱上进行上板验证；
5. 掌握从原理图设计、Verilog 编码、仿真验证到综合下载和硬件验证的完整实验流程，为后续 CPU 相关实验打下基础。

2 实验原理

实验指导手册中原始的定点加法实验实现的是两个 32 位数与 1 位低位进位输入的加法，主体模块 `adder.v` 的输入为 `operand1`、`operand2` 和 `cin`，输出为 `result` 和 `cout`。其实现方式是直接使用 Verilog 中的 “+” 运算符，由综合工具自动调用内部加法器资源完成实现。该方法结构简单，适合作为实验平台的入门设计。

本实验在此基础上进行扩展，将原始的两个 32 位数加法器改进为四个 32 位数加法器。改进后的主体模块输入为

$$operand1,\ operand2,\ operand3,\ operand4,\ cin$$

输出为

$$result[31:0],\ cout[1:0]$$

需要注意的是，四个 32 位无符号数与 1 位进位输入相加后的最大结果需要 **34 位**表示，因此不能继续沿用原始实验中单比特 `cout` 的设计，而应将结果拆分为低 32 位 `result[31:0]` 和高 2 位 `cout[1:0]`。即总和可表示为：

$$\{cout[1:0],\ result[31:0]\} = operand1 + operand2 + operand3 + operand4 + cin$$

为了保证计算时位宽正确，在实现中先将 4 个输入扩展为 34 位后再进行求和，从而避免高位进位在运算过程中被截断。

从系统结构上看，本实验仍采用 “主体功能模块 + 外围显示模块” 的分层设计思路。主体功能模块 `adder4_32` 负责完成四个 32 位输入数的加法运算；外围模块 `adder4_display` 负责调用主体加法器模块和 `lcd_module` 模块，通过实验箱上的触摸屏输入 32 位十六进制数据，并在 LCD 上显示四个输入操作数、最终计算结果以及高位进位信息。

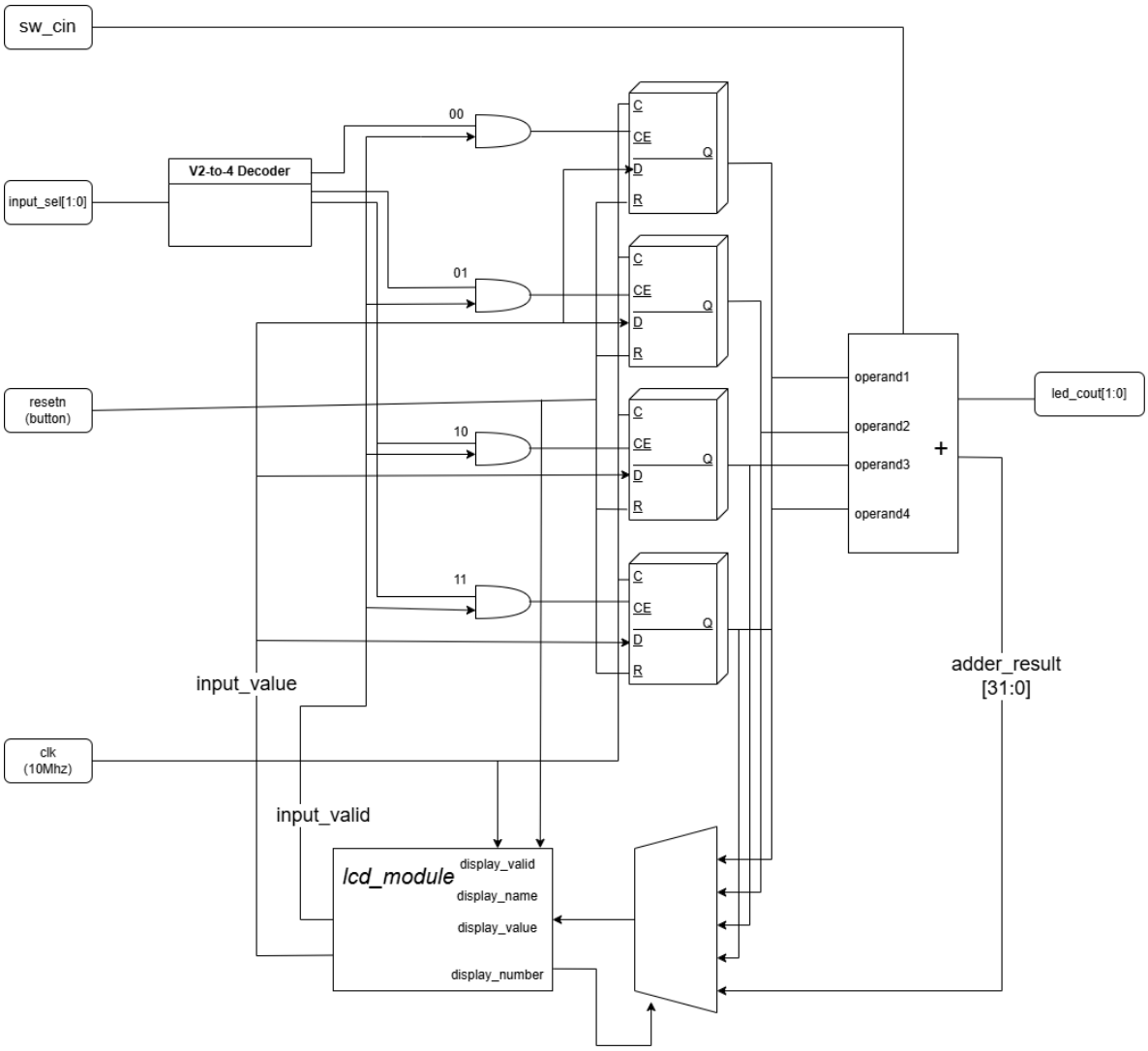


图 2.1: 四操作数加法器顶层模块框图

如图 2.1 所示，顶层模块 adder4_display 的输入包括低位进位输入 sw_cin、输入选择信号 input_sel[1:0]、复位信号 resetn 和时钟信号 clk。其中，input_sel[1:0] 进入 2-4 译码器后产生四路选择信号，分别控制四组寄存器的写使能，从而决定触摸屏输入值 input_value 当前写入哪一个操作数寄存器。

图中四组寄存器分别用于保存 operand1、operand2、operand3 和 operand4。各寄存器在时钟 clk 的控制下工作，并在复位信号 resetn 有效时清零。当 lcd_module 输出的 input_valid 有效时，触摸屏输入值会被写入由 input_sel[1:0] 当前选中的寄存器。

四组寄存器的输出作为 adder4_32 的四个操作数输入，同时 sw_cin 作为加法器的低位进位输入 cin。主体加法器模块完成四个 32 位数与 1 位进位输入的求和运算，输出低 32 位结果 adder_result[31:0] 和高 2 位进位 led_cout[1:0]，其中高 2 位进位直接输出到 LED，用于辅助观察实验结果。

为了在 LCD 上显示四个输入操作数以及运算结果，图中还设计了显示选择逻辑。该部分在 display_number 的控制下，从 operand1、operand2、operand3、operand4、adder_result 和 led_cout 中选择当前需要显示的数据，并分别送入 lcd_module 的 display_value、display_name 和 display_valid 接口。这样，LCD 模块即可循环显示 ADD1、ADD2、ADD3、ADD4、RESUL 和 COUT 等信息。

因此，本实验顶层模块的工作流程可以概括为：首先由 lcd_module 接收触摸屏输入数据，再通

过 input_sel[1:0] 将输入值写入对应的操作数寄存器；随后由 adder4_32 完成四个 32 位数与低位进位输入的求和运算；最后通过显示选择逻辑将输入数据、运算结果和高位进位送入 LCD 显示，并通过 LED 输出高位进位信息。

3 实验过程

3.1 adder.v 模块的修改

首先是对 adder.v 模块的修改，原代码实现的是两个 32 位二进制数的相加，因此我们对代码进行如下修改，基本思想是实现四个 32 位二进制数的相加。

原来的核心代码为：

```
assign {cout, result} = operand1 + operand2 + cin;
```

修改后的代码如下：

```
module adder(
    input  [31:0] operand1,
    input  [31:0] operand2,
    input  [31:0] operand3,    //【新增】
    input  [31:0] operand4,    //【新增】
    input         cin,
    output [31:0] result,
    output [1:0]  cout         //【修改】原来 cout 只有 1 位
);

wire [33:0] sum_all;           //【新增】用于保存四个 32 位数相加后的完整结果

assign sum_all = {2'b00, operand1}   //【修改】
               + {2'b00, operand2}   //【修改】
               + {2'b00, operand3}   //【新增】
               + {2'b00, operand4}   //【新增】
               + cin;

assign result = sum_all[31:0];       //【修改】
assign cout   = sum_all[33:32];      //【修改】

endmodule
```

这里由于四个 32 位二进制数相加后有可能产生两位进位，因此将 cout 由原来的 1 位改为 2 位。

3.2 仿真文件的编写与测试

编写完 adder.v 模块之后，我们编写仿真文件对模块进行测试。原来的仿真文件只对两个操作数和进位信号 cin 进行测试，因此这里增加 operand3 和 operand4 两个输入。

修改后的仿真文件如下：

```
module adder_tb;
```

```
2
3  reg   [31:0] operand1;
4  reg   [31:0] operand2;
5  reg   [31:0] operand3;     //【新增】
6  reg   [31:0] operand4;     //【新增】
7  reg          cin;
8
9  wire  [31:0] result;
10 wire  [1:0]  cout;         //【修改】原来 cout 为 1 位
11
12 adder uut(
13     .operand1(operand1),
14     .operand2(operand2),
15     .operand3(operand3), //【新增】
16     .operand4(operand4), //【新增】
17     .cin(cin),
18     .result(result),
19     .cout(cout)
20 );
21
22 initial begin
23     operand1 = 0;
24     operand2 = 0;
25     operand3 = 0;         //【新增】
26     operand4 = 0;         //【新增】
27     cin      = 0;
28 end
29
30 always #10 operand1 = $random;
31 always #10 operand2 = $random;
32 always #10 operand3 = $random;    //【新增】
33 always #10 operand4 = $random;    //【新增】
34 always #10 cin      = $random % 2;
35
36 endmodule
```

在仿真过程中，四个操作数及进位信号 `cin` 均为随机生成。通过仿真波形可以验证，修改后的模块能够正确实现四个 32 位二进制数的加法。

3.3 lcd_module 模块的导入

然后我们将原本预设好的 `lcd_module` 模块导入，为后续在 LCD 屏幕上显示输入数据和结果做准备。

3.4 顶层模块 adder_display.v 的改写

接着我们对顶层模块 `adder_display.v` 进行改写。

现在有四个操作数，因此在输入 `input_sel` 时需要用两位二进制数进行表示：

- 00 表示输入第一个数
- 01 表示输入第二个数
- 10 表示输入第三个数
- 11 表示输入第四个数

同时，对于四个 32 位二进制数的加法，有可能出现两位的进位，因此将 led_cout 也改为两位二进制数。

顶层模块中端口的修改如下：

```
input  [1:0] input_sel,      //【修改】原来为 1 位
input        sw_cin,
output [1:0] led_cout        //【修改】原来为 1 位
```

在调用加法器模块时，需要输入四个 32 位二进制数，因此将所有操作数都加入模块中：

```
adder u_adder(
    .operand1(adder_operand1),
    .operand2(adder_operand2),
    .operand3(adder_operand3),    //【新增】
    .operand4(adder_operand4),    //【新增】
    .cin(sw_cin),
    .result(adder_result),
    .cout(adder_cout)
);

assign led_cout = adder_cout;     //【修改】
```

在输入判断部分，也需要增加对 operand3 和 operand4 的输入控制。例如：

```
always @(posedge clk) begin
    if (!resetn)
        adder_operand1 <= 32'd0;
    else if (input_valid && input_sel == 2'b00)
        adder_operand1 <= input_value;
end

always @(posedge clk) begin
    if (!resetn)
        adder_operand2 <= 32'd0;
    else if (input_valid && input_sel == 2'b01)
        adder_operand2 <= input_value;
end

always @(posedge clk) begin
    if (!resetn)
        adder_operand3 <= 32'd0;              //【新增】
    else if (input_valid && input_sel == 2'b10)
        adder_operand3 <= input_value;        //【新增】
```

```
end

always @(posedge clk) begin
    if (!resetn)
        adder_operand4 <= 32'd0;                //【新增】
    else if (input_valid && input_sel == 2'b11)
        adder_operand4 <= input_value;          //【新增】
end
```

这样，如果拨码开关拨向 10 和 11，将分别进行 operand3 和 operand4 的输入。

在 LCD 屏幕显示部分，也要增加对 operand3 和 operand4 的显示，例如：

```
always @(posedge clk) begin
    case(display_number)
        6'd1: begin
            display_valid <= 1'b1;
            display_name  <= "ADD1_";
            display_value <= adder_operand1;
        end
        6'd2: begin
            display_valid <= 1'b1;
            display_name  <= "ADD2_";
            display_value <= adder_operand2;
        end
        6'd3: begin
            display_valid <= 1'b1;
            display_name  <= "ADD3_";           //【新增】
            display_value <= adder_operand3;    //【新增】
        end
        6'd4: begin
            display_valid <= 1'b1;
            display_name  <= "ADD4_";           //【新增】
            display_value <= adder_operand4;    //【新增】
        end
        6'd5: begin
            display_valid <= 1'b1;
            display_name  <= "RESUL";
            display_value <= adder_result;
        end
        6'd6: begin
            display_valid <= 1'b1;
            display_name  <= "COUT_";           //【新增】
            display_value <= {30'd0, adder_cout};//【新增】
        end
        default: begin
            display_valid <= 1'b0;
            display_name  <= 40'd0;
            display_value <= 32'd0;
        end
```

```
38      endcase
39  end
```

这样 LCD 屏幕就能够显示 operand3 和 operand4，同时也能够显示最后的结果和进位。

3.5 约束文件的修改

接下来完成对约束文件的修改。我们用到实验箱的接口为拨码开关 1、2、8，其中 1 号和 2 号拨码开关用于表示输入的操作数，8 号拨码开关用于表示输入的进位信号；LED 灯用到 1 号和 2 号来表示输出的进位信号。

由于 input_sel 改为了 2 位，led_cout 也改为了 2 位，因此约束文件也需要相应修改。修改后的相关代码如下：

```
1
2  set_property PACKAGE_PIN H7    [get_ports {input_sel[1]}]    ;# 【修改】
3  set_property PACKAGE_PIN D5    [get_ports {input_sel[0]}]    ;# 【修改】
4  set_property PACKAGE_PIN AC21  [get_ports sw_cin]
5
6  set_property PACKAGE_PIN AC24  [get_ports {led_cout[1]}]    ;# 【修改】
7  set_property PACKAGE_PIN AC22  [get_ports {led_cout[0]}]    ;# 【修改】
```

修改完成后重新生成比特流文件并下载到实验箱中，最终可以通过拨码开关输入四个操作数和进位信号，并在 LCD 和 LED 上正确显示运算结果，说明设计是正确的。

4 实验结果

根据实验指导手册要求，本实验分别通过行为级仿真和实验箱上板两种方式对改进后的四操作数加法器进行验证。仿真部分用于检查功能逻辑是否正确，上板部分用于验证设计在 FPGA 实验平台上的实际运行情况。

4.1 波形仿真结果

在 Vivado 中对 adder4_tb 进行行为级仿真，重点观察 operand1、operand2、operand3、operand4、cin、result 以及 cout 信号。按照手册实验一中的建议，将波形窗口中的数据显示格式改为十六进制，以便于观察 32 位操作数与结果。

图 4.2 给出了 43ns 时刻附近的波形结果，图 4.3 给出了 94ns 时刻附近的波形结果。

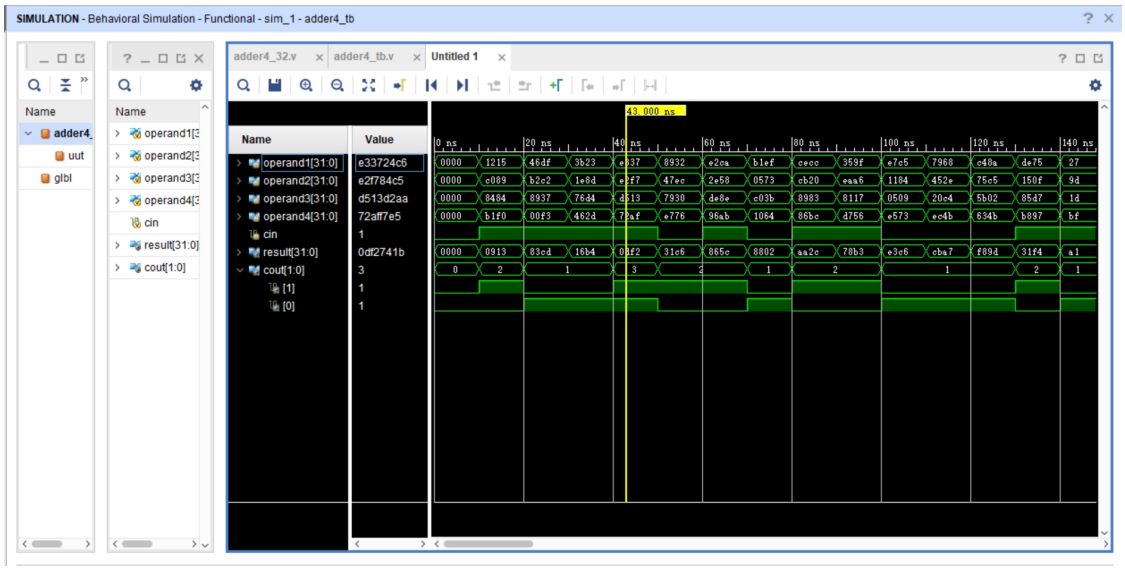


图 4.2: 43ns 时刻附近的四操作数加法器仿真波形

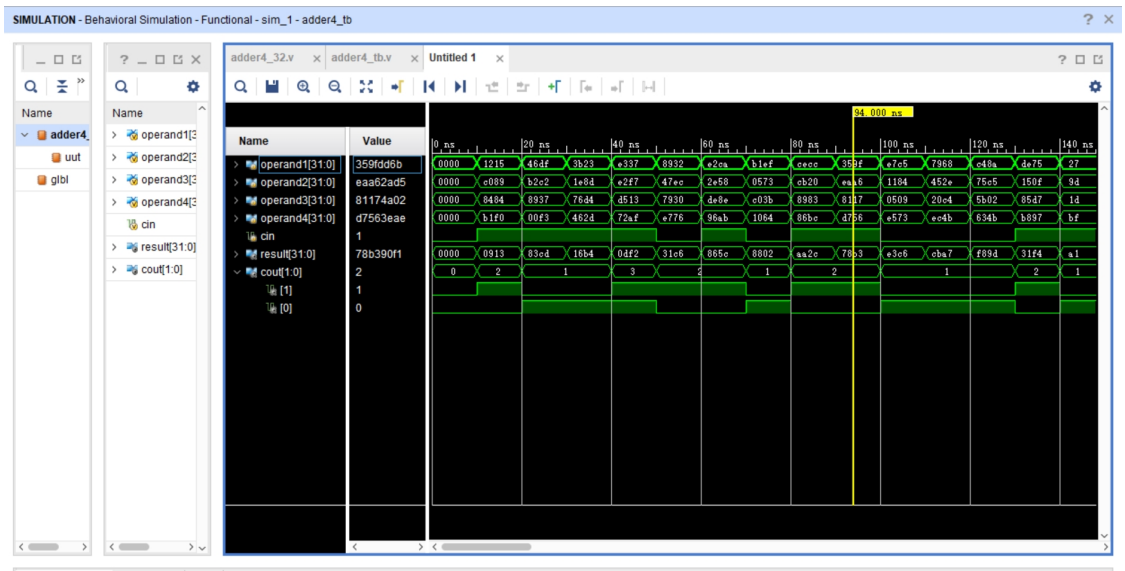


图 4.3: 94ns 时刻附近的四操作数加法器仿真波形

从图中可以看出，随着四个输入操作数和低位进位输入的变化，输出信号 `result` 与 `cout` 也能够实时更新，说明加法器能够正确响应输入变化并完成四操作数求和。

4.2 波形数据分析

为了验证仿真结果的正确性，下面分别对图 4.2 和图 4.3 中选取的两个时刻进行分析。

(1) 43ns 时刻数据分析 由图 4.2 可读出该时刻各输入输出信号为:

- `operand1 = 0xe33724c6`
- `operand2 = 0xe2f784c5`

- operand3 = 0xd513d2aa
- operand4 = 0x72aff7e5
- cin = 1
- result = 0x0df2741b
- cout = 3

理论计算结果为：

$$0xe33724c6 + 0xe2f784c5 + 0xd513d2aa + 0x72aff7e5 + 1 = 0x30df2741b$$

因此高 2 位应为：

$$cout = 3$$

低 32 位应为：

$$result = 0x0df2741b$$

这与波形图中显示的结果完全一致，说明在该组输入下加法器输出正确。

（2）94ns 时刻数据分析 由图 4.3 可读出该时刻各输入输出信号为：

- operand1 = 0x359fdd6b
- operand2 = 0xeaa62ad5
- operand3 = 0x81174a02
- operand4 = 0xd7563eae
- cin = 1
- result = 0x78b390f1
- cout = 2

理论计算结果为：

$$0x359fdd6b + 0xeaa62ad5 + 0x81174a02 + 0xd7563eae + 1 = 0x278b390f1$$

因此高 2 位应为：

$$cout = 2$$

低 32 位应为：

$$result = 0x78b390f1$$

该结果与仿真波形中的输出值一致，进一步说明改进后的四操作数加法器仿真功能正确。
综合上述两组波形分析可知，四个 32 位输入操作数与 1 位低位进位输入满足如下关系：

$$\{cout, result\} = operand1 + operand2 + operand3 + operand4 + cin$$

即设计的逻辑功能满足实验要求。

4.3 实验箱上板结果

在完成功能仿真后，继续按照实验指导手册流程将设计综合、实现并生成 bit 文件，再下载到 FPGA 实验箱进行上板验证。上板时通过 `input\_sel[1:0]` 依次选择 4 个输入操作数，通过 LCD 触摸屏输入 32 位十六进制数据，并通过 `sw\_cin` 控制低位进位输入。

本实验选取如下两组典型情况进行上板测试：

1. 四个操作数分别输入 `1,2,3,4`，且 `cin=0`；
2. 四个操作数保持不变，仅将 `cin` 由 0 改为 1。

实验箱显示结果分别如图 4.4 和图 4.5 所示。



图 4.4: 实验箱验证结果（$cin = 0$）



图 4.5: 实验箱验证结果（$cin = 1$）

4.4 实验箱结果分析

对于图 4.4，LCD 上显示为：

- `ADD1 : 00000001`
- `ADD2 : 00000002`
- `ADD3 : 00000003`
- `ADD4 : 00000004`
- `RESUL: 0000000A`
- `COUT : 00000000`

此时理论结果为：

$$1 + 2 + 3 + 4 = 10 = 0x0000000A$$

可以看到 LCD 显示值与理论值完全一致，说明当 `cin=0` 时电路工作正确。

对于图 4.5，保持四个输入操作数不变，仅将低位进位输入改为 `cin=1`。此时 LCD 显示为：

- ADD1 : 00000001
- ADD2 : 00000002
- ADD3 : 00000003
- ADD4 : 00000004
- RESUL: 0000000B
- COUT : 00000000

理论结果为:

$$1+2+3+4+1=11=0x0000000B$$

从图中可以看到，RESUL 由 0000000A 变为 0000000B，恰好增加 1，说明低位进位输入 cin 已正确参与运算。由于本组测试数据没有产生超过 32 位的高位进位，因此 COUT 仍显示为 00000000，这同样与理论分析一致。

综合行为级仿真结果与实验箱上板结果可知，本实验设计的四个 32 位数加法器能够正确完成加法运算，既满足功能仿真要求，也满足 FPGA 硬件验证要求。

5 实验收获

本次实验是对上学期 Verilog 知识的一次巩固，使我进一步理解了加法器的原理，熟悉了 Verilog 的语法和多模块工程的编写方法。通过将原来的两个 32 位数加法扩展为四个 32 位数相加，我认识到在数字电路设计中，结果位宽和进位位数都需要经过分析后确定，不能直接沿用原有设计。

在实验过程中，我掌握了使用 testbench 对功能模块进行仿真的基本方法，并学会结合波形图对结果进行分析和验证。同时，在顶层模块、显示模块和约束文件的修改过程中，我对功能模块与硬件接口之间的关系有了更清晰的认识。

通过本次实验，我完整经历了代码编写、仿真验证、上板测试的全过程，对 FPGA 实验流程有了更加直观和完整的理解，也为后续更复杂的数字系统设计实验打下了基础。